

ASSIGNMENT - 4

Azure Cloud Fundamentals and Data Pipeline Implementation using ADF

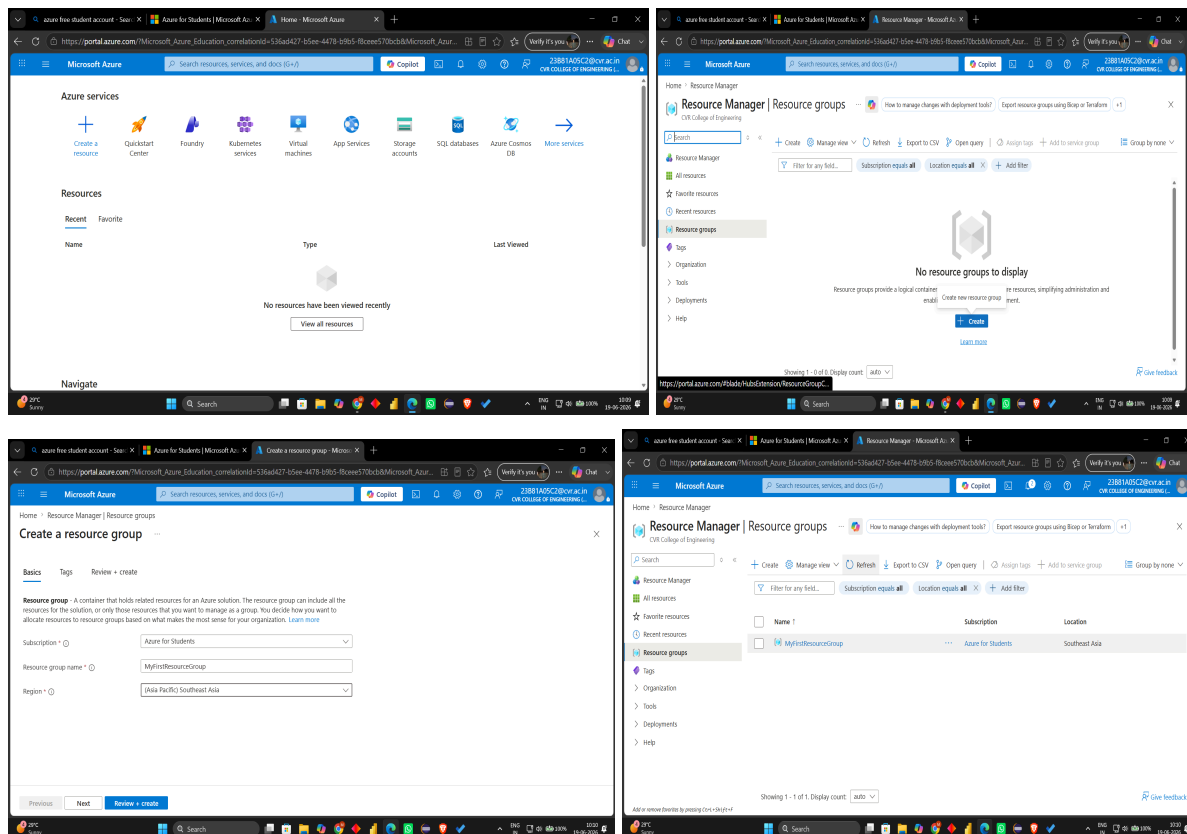
Submitted By

Name: Mohammed Sameer

College: CVR College of Engineering

Step 1: Create a Resource Group

Screenshot



Explanation

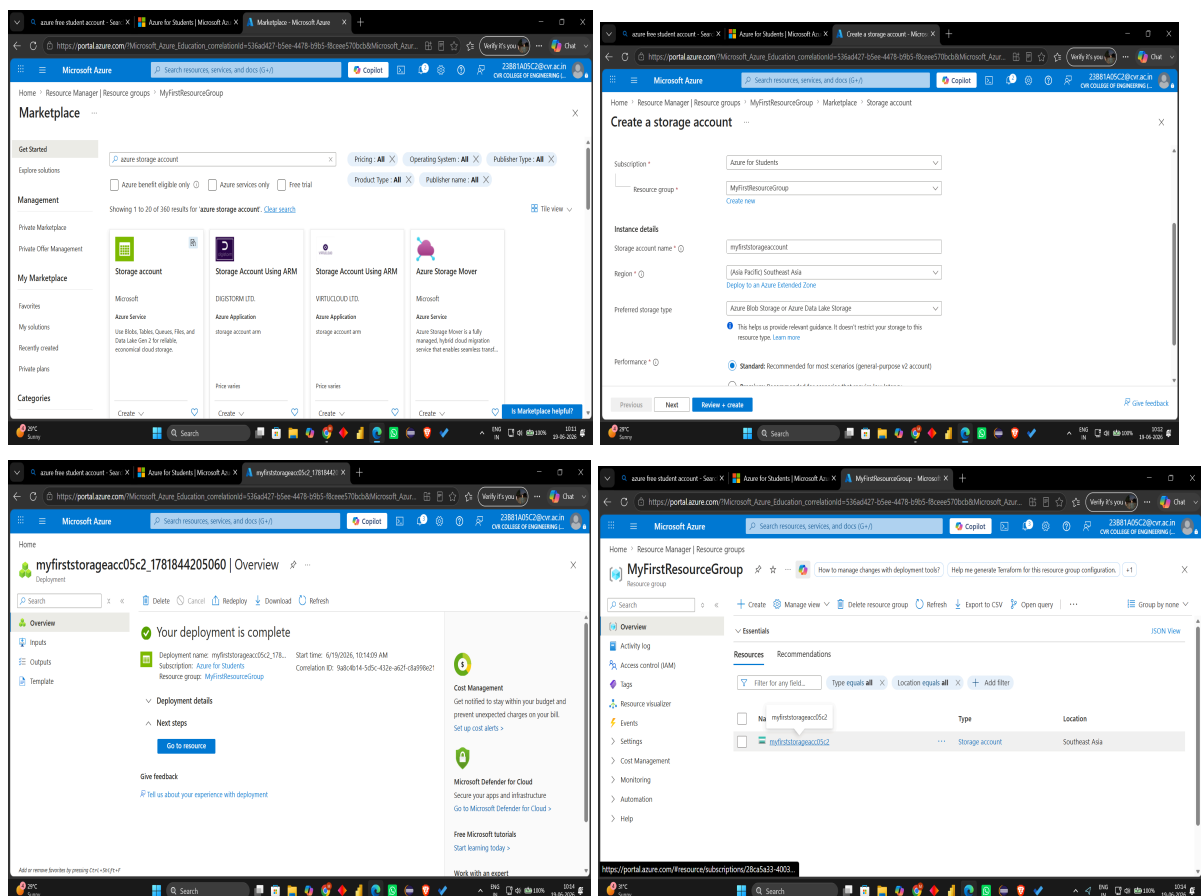
- Open the Azure Portal.
- Search for Resource Groups.
- Click Create.
- Enter a resource group name and region.
- Click Review + Create and create the resource group.

Purpose:

A Resource Group is where we create all our resources like storage account, adf etc.

Step 2: Create Storage Account and Blob Container

Screenshot



Explanation

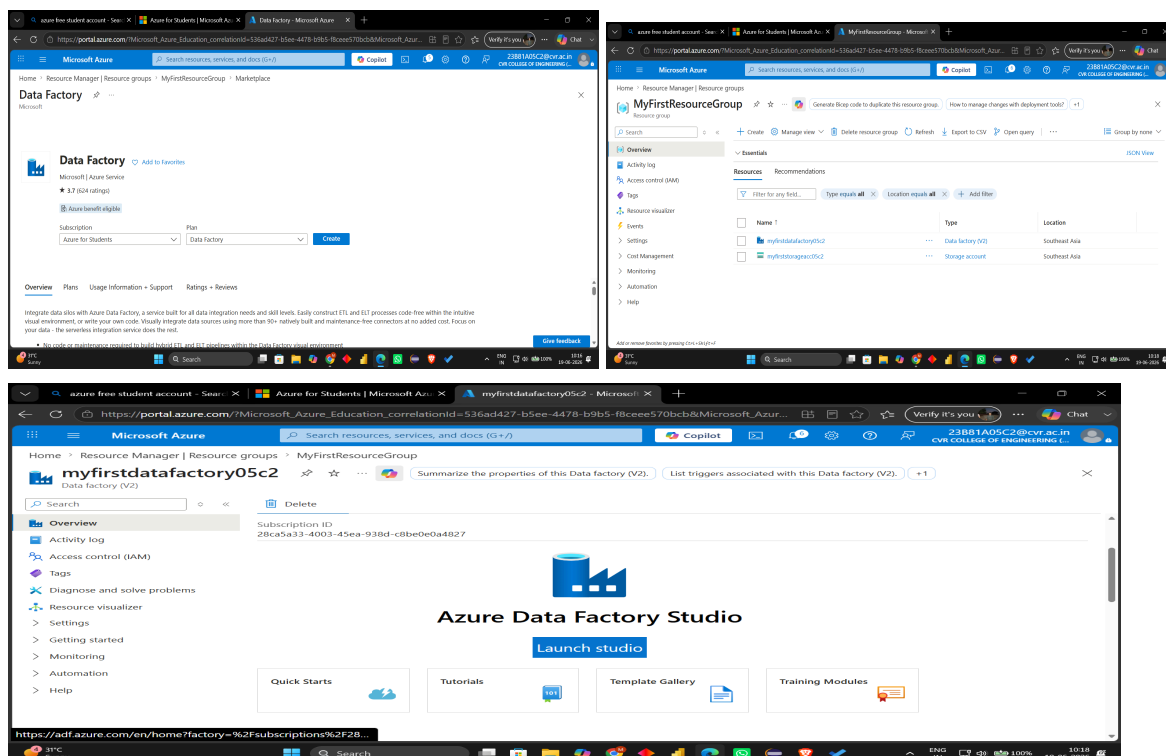
- Open **Storage Accounts** and click **Create**.
- Select the resource group.
- Provide a storage account name.
- Create the storage account.
- Inside the storage account, create a Blob container.
- Upload the CSV file to the container.

Purpose:

Blob Storage stores files such as CSV, images, and documents.

Step 3: Create Azure Data Factory

Screenshot



Explanation

- Search for **Azure Data Factory**.
- Click **Create**.
- Select the resource group.
- Enter a Data Factory name.

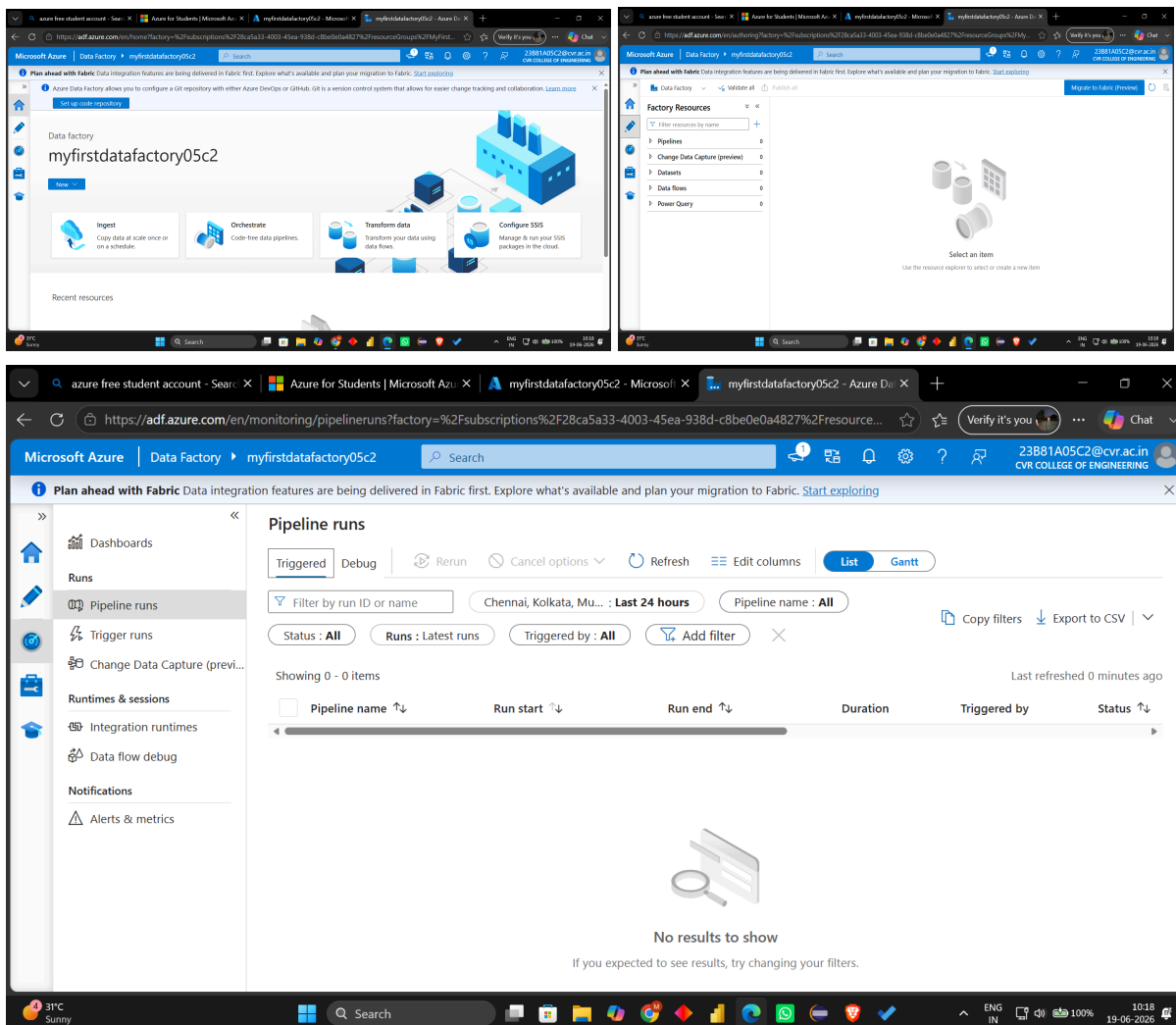
- Deploy the service successfully.

Purpose:

Azure Data Factory is used to move and transform data between different sources.

Step 4: Explore Azure Data Factory Studio

Screenshot



Explanation

Azure Data Factory contains three main sections:

Author

Used to create pipelines, datasets, and activities.

Monitor

Used to check pipeline execution status.

Manage

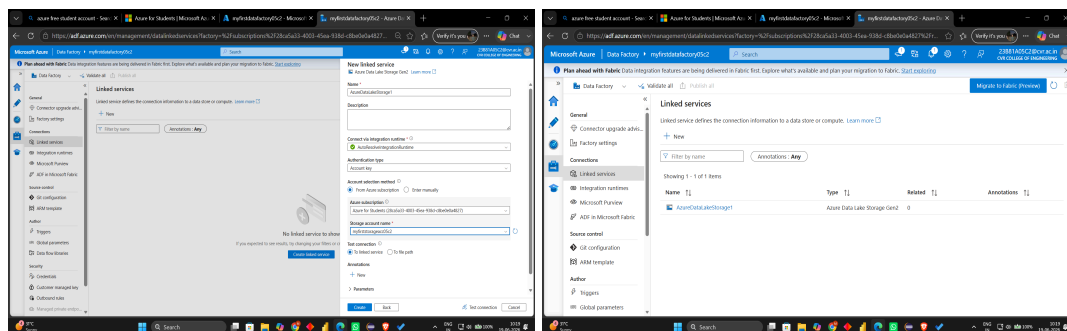
Used to create linked services and manage connections.

Purpose:

These sections help in building and monitoring data pipelines.

Step 5: Create Linked Service

Screenshot



Explanation

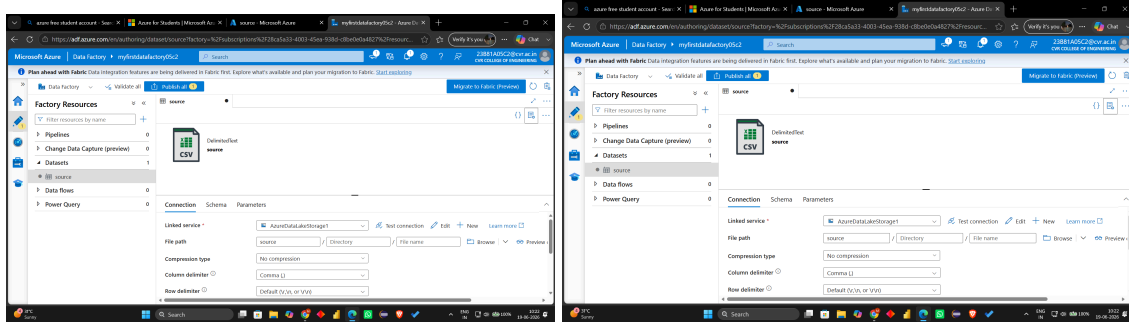
- Open the **Manage** section.
- Select **Linked Services**.
- Create a connection to Azure Blob Storage.
- Test the connection.
- Save the linked service.

Purpose:

A Linked Service acts like a connection between Azure Data Factory and Storage Account.

Step 6: Create Source and Destination Datasets

Screenshot



Explanation

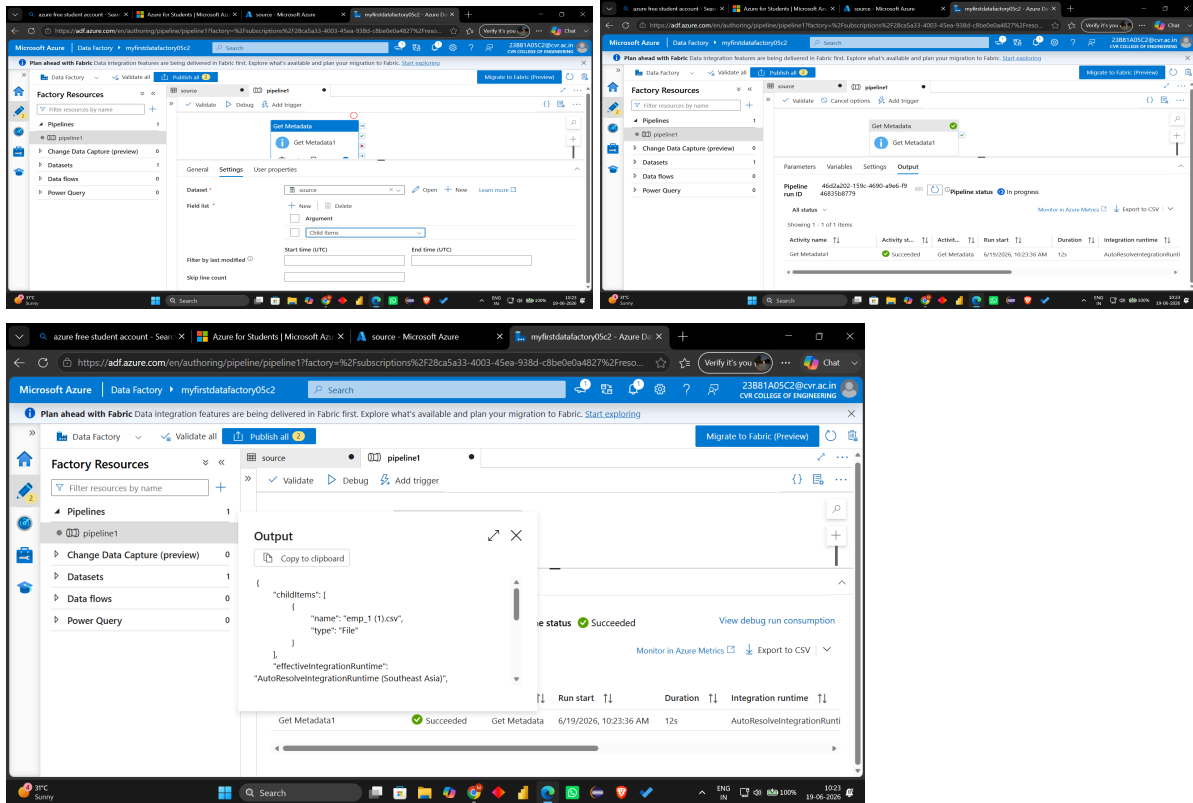
- Create a source dataset for the input CSV file.
- Create a destination dataset for the output file.
- Connect both datasets to Blob Storage.

Purpose:

Datasets tell Azure Data Factory where data should be read from and where it should be stored.

Step 7: Use Get Metadata Activity

Screenshot



Explanation

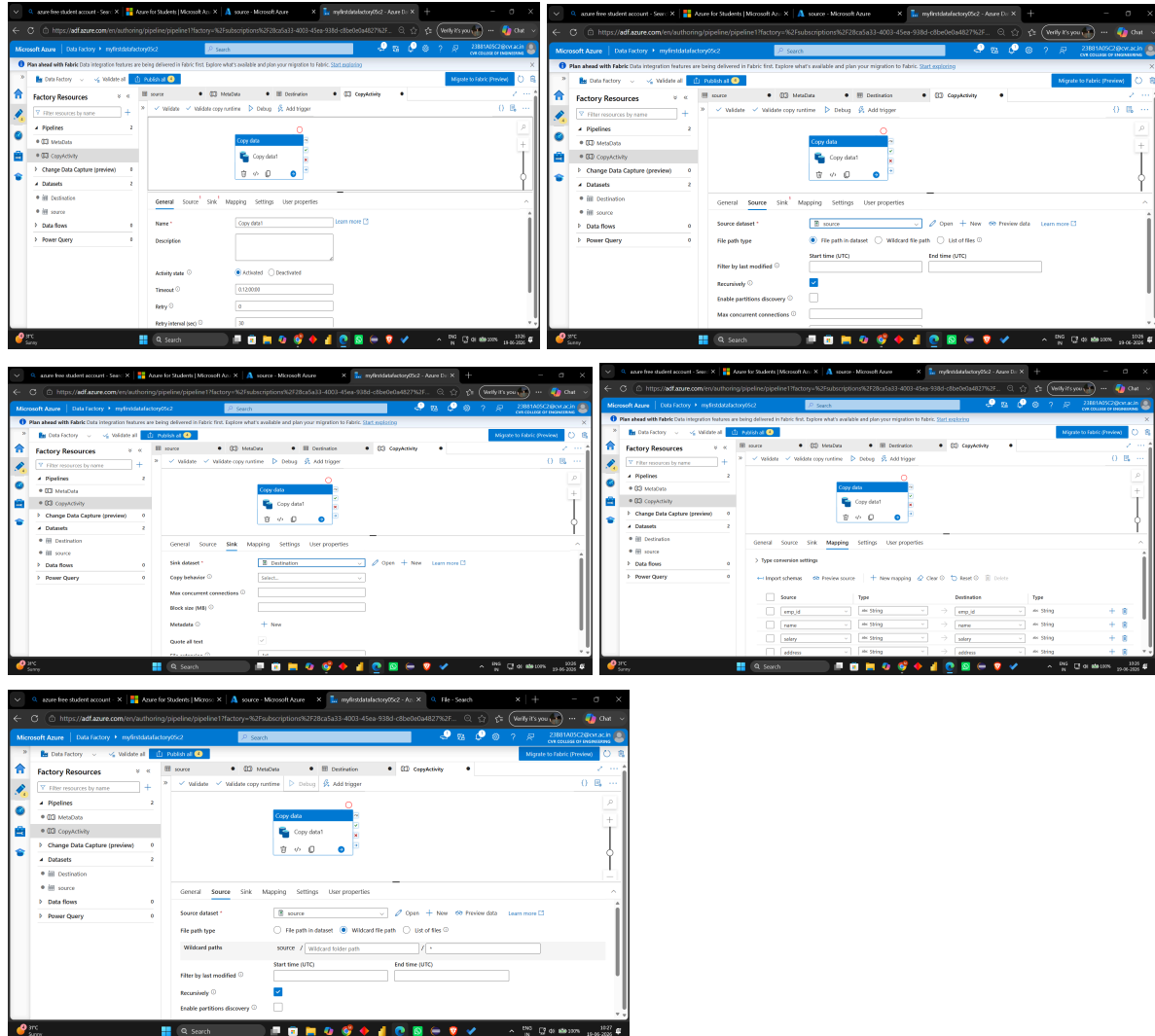
- Add the **Get Metadata** activity to the pipeline.
- Select the source dataset.
- Choose properties like file name and size.
- Execute the activity.

Purpose:

Get Metadata retrieves information about the file without reading the file contents.

Step 8: Build Pipeline Using Copy Data Activity

Screenshot



Explanation

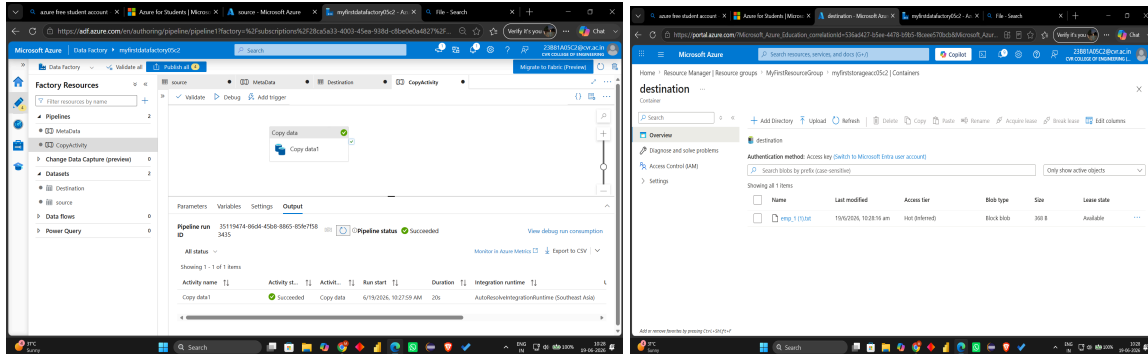
- Add the **Copy Data** activity.
- Configure the source dataset.
- Configure the destination dataset.
- Connect the activities properly.
- Validate the pipeline.

Purpose:

Copy Data activity transfers files from the source location to the destination location

Step 9: Execute and Monitor the Pipeline

Screenshot



Explanation

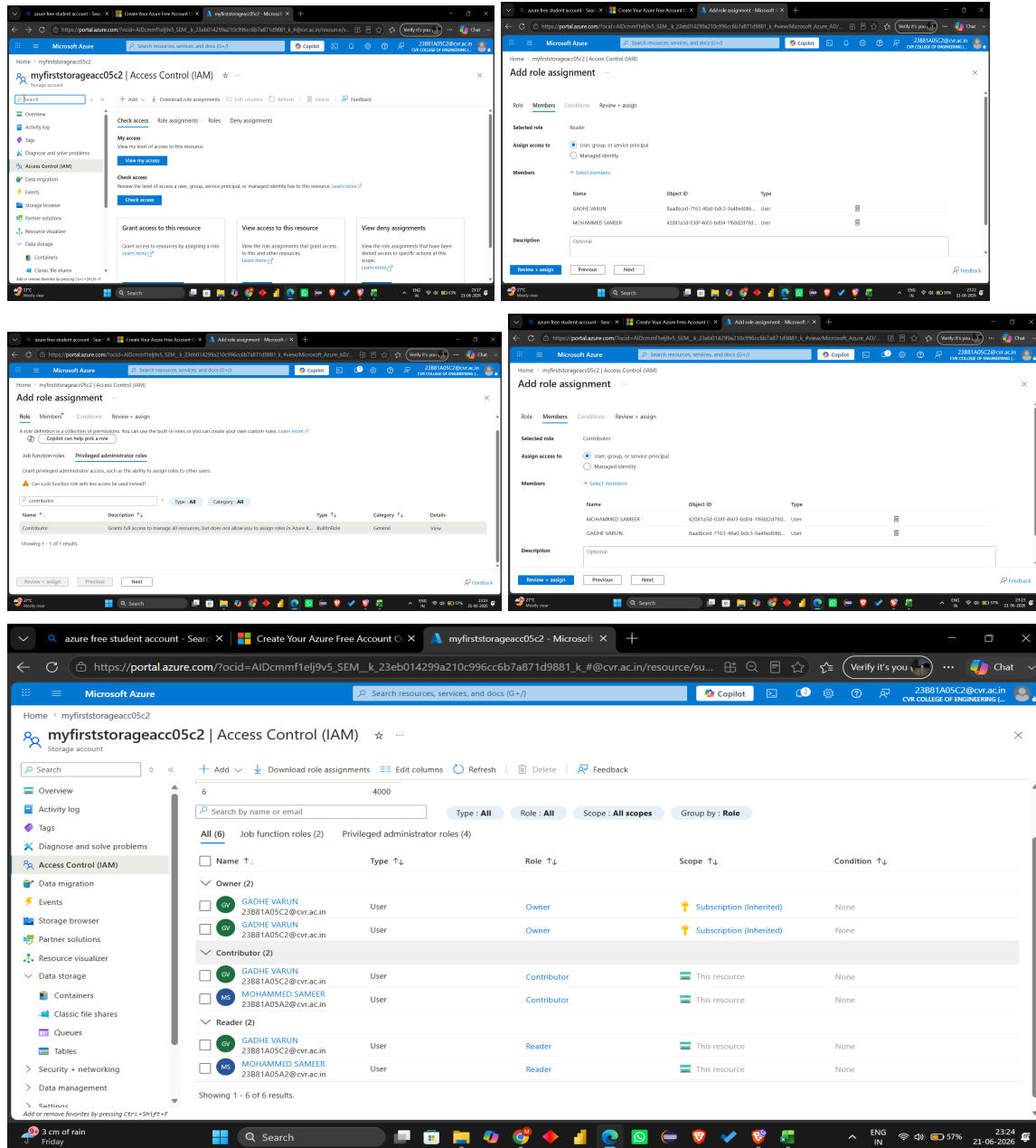
- Click **Debug** to test the pipeline.
- Use **Trigger Now** to execute it.
- Open the **Monitor** section.
- Verify whether the pipeline ran successfully.

Purpose:

Monitoring helps track the execution status and identify errors.

Step 10: Assign IAM Roles

Screenshot



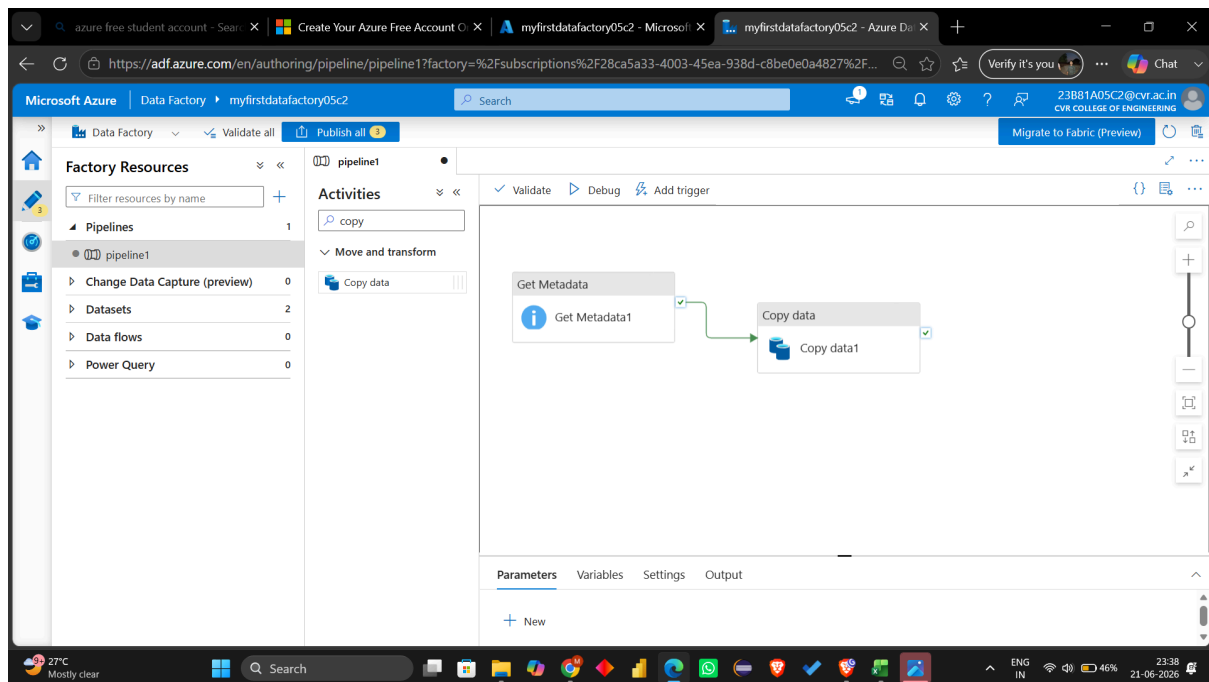
Explanation

- Open **Access Control (IAM)**.
- Assign **Reader** role for viewing resources.
- Assign **Contributor** role for managing resources.
- Ensure Azure Data Factory has access to Storage Account.

- **Purpose:**
IAM roles provide permissions and secure access to Azure resources.

Step 11: Implement End-to-End Pipeline

Screenshot



Explanation

- Data is stored in the source Blob container.
- Azure Data Factory reads the source file.
- Metadata is checked using Get Metadata activity.
- Copy Data activity transfers the file.
- Output file is stored in the destination container.

Data Flow:

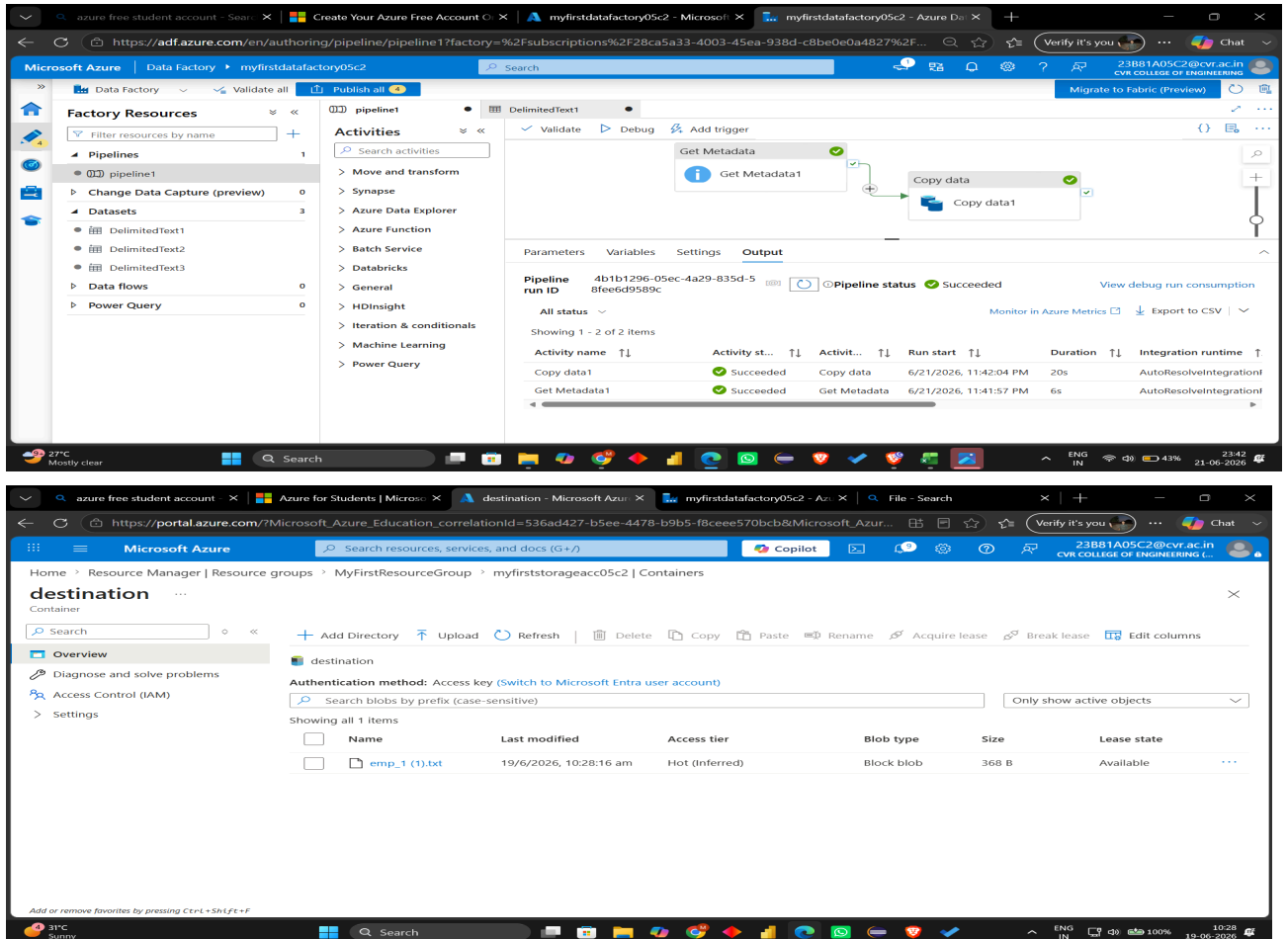
Source Blob Storage → Azure Data Factory → Destination Blob Storage

Purpose:

This completes the end-to-end data movement process.

Pipeline Execution Result

Screenshot

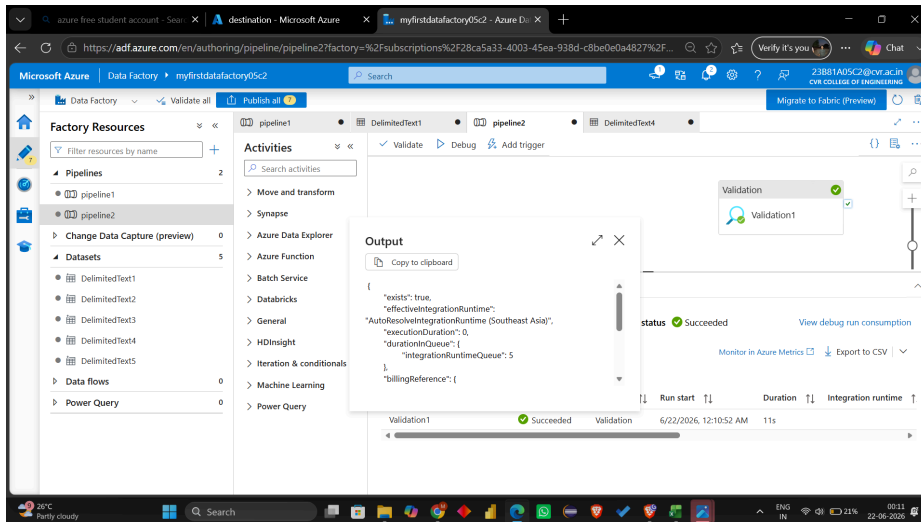


Explanation

- The pipeline executed successfully.
- The CSV file was copied from source to destination.
- Metadata validation confirmed the file information.
- Pipeline status showed **Succeeded**.

Step 12: Validate File Before Data Transfer

Screenshot



Explanation

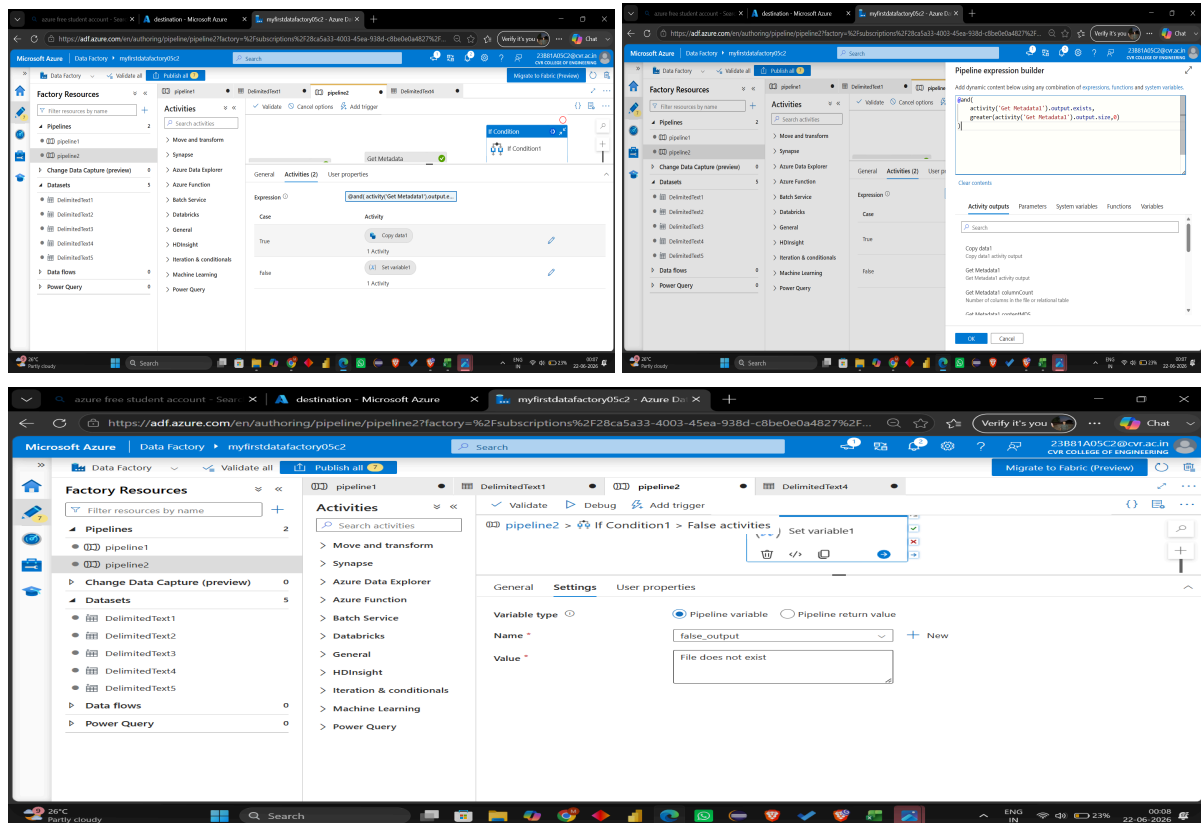
- Add a **Validation Activity** to the pipeline.
- Configure it to check whether the source CSV file exists in Blob Storage.
- Add a **Get Metadata Activity** to retrieve file information.
- Select properties such as **Exists** and **Size**.
- Add an **If Condition Activity** to verify that the file size is greater than 0 bytes.
- If the condition is true, execute the **Copy Data Activity**.
- If the condition is false, execute a **Fail Activity** or send an alert.

Purpose:

This validation process ensures that only valid and non-empty files are copied to the destination.

Step 13: Configure If Condition Expression

Screenshot



Explanation

- Open the **If Condition Activity**.
- Use the following expression to check whether the file exists and its size is greater than zero.

```
@and(  
    activity('Get Metadata1').output.exists,  
    greater(activity('Get Metadata1').output.size,0)  
)
```

- The expression returns **True** if the file exists and contains data.
- The expression returns **False** if the file is empty or unavailable.

Purpose:

The If Condition activity controls the pipeline flow and prevents empty files from being processed.

Step 14: Complete Pipeline Flow

Screenshot

The top screenshot shows the 'Design' view of a pipeline named 'pipeline2' in the Microsoft Azure Data Factory portal. The pipeline flow is as follows:

```
graph LR; Validation[Validation1] --> GetMetadata[Get Metadata1]; GetMetadata --> IfCondition[If Condition1]; IfCondition -- True --> CopyData[Copy data1]; IfCondition -- False --> SetVariable[Set variable1];
```

The bottom screenshot shows the 'Monitor' view of the same pipeline. It displays a table of activity runs with columns for Activity name, Activity status, Activity name, Run start, Duration, and Integration runtime. The activities listed are Copy data1, If Condition1, Get Metadata1, and Validation1, all with a status of 'Succeeded'.

Activity name	Activity st...	Activit...	Run start	Duration	Integration runtime
Copy data1	Succeeded	Copy data	6/22/2026, 12:03:08 AM	18s	AutoResolveIntegrationI
If Condition1	Succeeded	If Condition	6/22/2026, 12:03:08 AM	20s	
Get Metadata1	Succeeded	Get Metadata	6/22/2026, 12:03:01 AM	6s	AutoResolveIntegrationI
Validation1	Succeeded	Validation	6/22/2026, 12:02:49 AM	12s	

Explanation

The complete pipeline consists of the following activities:

1. Validation Activity

- Checks whether the source file is available.
- 2. **Get Metadata Activity**
 - Retrieves information such as file existence and file size.
- 3. **If Condition Activity**
 - Checks whether the file size is greater than 0 bytes.
- 4. **Copy Data Activity**
 - Copies the file from the source Blob container to the destination Blob container.
- 5. **Fail Activity**
 - Stops the pipeline if the file is missing or empty.

Pipeline Flow:

Start



Validation Activity



Get Metadata Activity



If Condition (File Size > 0)



True → Copy Data Activity



False → Fail Activity

Purpose:

This workflow improves data quality and ensures reliable data transfer.
